Verifier-Anchored RL over a Frozen LLM

Design, prior art, and first live results

Alexander Towell

July 2026

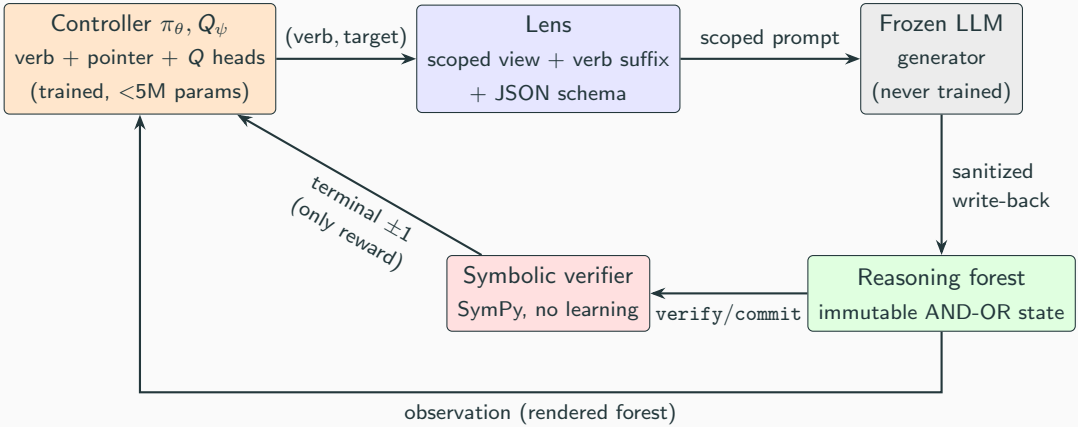
What this is (and is not)

The system

A tiny external controller (verb head + pointer head + Q head, a few million parameters) learns to orchestrate a frozen LLM by issuing typed (verb, target) actions over a structured, nested reasoning forest. A symbolic verifier is the only reward source.

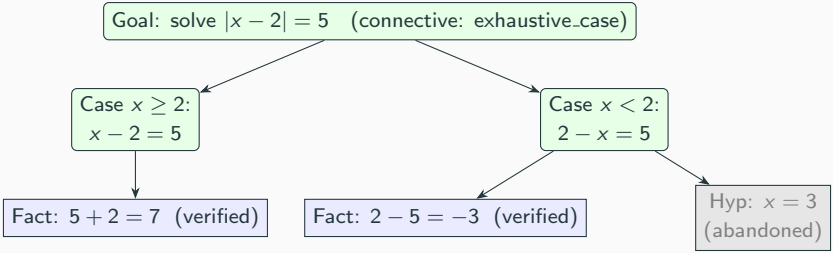
- **Not** o1 / DeepSeek-R1: RL never touches the LLM's weights
- **Not** a learned reward model: no PRM, ORM, critic, or LLM-judge anywhere in the reward path
- Training spine: off-policy value-based Q-learning; MCTS is a strictly additive accelerator and the test-time compute knob

Architecture: one action selects the operation and scopes the context



`verify`/`commit`/`abandon` make no LLM call: the policy can neither author what is checked nor what it is checked against.

State: a nested AND-OR reasoning forest



- Frozen dataclasses: `Goal`, `Hypothesis`, `Fact`, `VerifierCall`; verbs are total functions Graph $\rightarrow$ Ok | Err
- No reward or value stored in nodes; `solved`/`verified` are derived predicates
- Structure never gates: the connective is metadata for the controller, not a hard rule
- Closed subtrees compact to one line in every rendered view (built-in roll-back / summarize)

Action space: six verbs, one pointer

Verb	LLM call?	Effect
`split`	yes	decompose a goal into subgoals or exhaustive cases
`form_hypothesis`	yes	add a tentative claim under a goal
`compute`	yes	add one checkable ground equation as a fact
`verify`	no	run the symbolic checker on a node's statement
`commit`	no	close a goal on verifier-vouched evidence
`abandon`	no	prune a dead end (excluded from menus and views)

The lens property

Choosing (verb, target) simultaneously picks the operation *and* the slice of state the frozen generator sees: problem + ancestor path + target subtree, siblings hidden, closed subtrees compacted. Write-back is local and sanitized against marker injection.

Reward invariant and the training ladder

Reward (load-bearing)

- Terminal $+1/-1$ at root `commit`; 0 elsewhere
- Masked/illegal action: small penalty
- No learned reward of any kind, ever
- Aesthetic signals may enter only as search priors, never as Bellman targets

Three rungs

1. LLM-as-controller: a strong teacher picks $(\text{verb}, \text{target})$; JSONL trajectories logged
2. Behavior cloning: warm-start π_θ on rung-1 data
3. Q-learning fine-tune: off-policy, n-step / MC returns from the verifier; MCTS optional for better targets

Design-true collection regime: API teacher decides, a *local small generator* writes all content, so trajectories reflect the deployed model's state distribution.

Prior art: the landscape

Cluster	Exemplars	How we differ
Search + train the LLM (distill / fine-tune)	TS-LLM, GSoS, OREO, Agent Q, rStar-Math	LLM frozen; controller is external and tiny
Learned process reward	ReST-MCTS*, rStar-Math, V-STaR, TS-LLM (ORM)	symbolic verifier is the *only* reward
Prompt-level RL over frozen LM	RLPrompt, TEMPERA	multi-step typed ops + pointer over a forest
External module + frozen LLM	PNLC (value only), UniR (logit level)	discrete action policy, operation-level Q
Context control (concurrent)	Active Context Curation (2026)	7B pruning policy vs $<$5M lens-as-byproduct

Prior art: the three real threats, and what survives

Threats a reviewer will raise

1. TS-LLM read as "same loop, different reward source"
2. ArCHer + GSoS + RLVR: "each ingredient is published, the combination is engineering"
3. Active Context Curation as concurrent work on the lens axis

What survives: the conjunction, claimed as such

- Tiny *external* controller (neither the LLM nor an LLM-sized critic)
- Factored verb + pointer heads over live node ids of a growing forest
- Off-policy value-based Q-learning as the spine, MCTS strictly additive
- Symbolic verifier as the sole reward, no learned value/PRM/ORM anywhere
- The lens property: action selection = context scoping, with local write-back

Honest caveat: defensible but thin per-element; claim the conjunction, not a new RL algorithm.

Rung 1 is live: teacher-as-controller on GSM8K

Setup

- gpt-4.1-mini as both controller-teacher and generator
- 247 GSM8K problems, depth limit 16, 3 workers, hard token budget
- Verifier: SymPy; ground equations only

Headline numbers

- 71.7% solved (177/247)
- 1,942 decisions; 7 fallbacks (0.36%)
- median 7 turns; pilot cost $< \$0.20$

Verb usage (1,942 decisions)

`compute`	1165
`verify`	558
`commit`	203
`split`	7
`form_hypothesis`	5
`abandon`	4

The verify-iterate loop works: failed checks are surfaced as `(verify failed xN)` and the teacher reformulates instead of retrying.

Diagnosis: healthy linear skill, collapsed structure

- The dominant trajectory is `compute` → `verify` → `commit`: correct play, because GSM8K is linearly solvable
- Structural verbs (`split`, `abandon`) appear in under 1% of decisions: behavior cloning on this data alone would learn a linear-chain policy
- This is a *dataset* property, not a system failure: the AND-OR forest never earns its keep on problems with no cases

The fix: generate our own problems

The verifier can *decide* answers (SymPy solves the task), so we can generate problems in reverse: choose the case structure and answer set first, synthesize the statement around it. Ground truth by construction, unlimited scale, difficulty as a curriculum knob. Absolute values, piecewise casework, inequalities, "find all solutions": each makes `split` and `abandon` genuinely optimal.

Next steps

1. **Finish the GSM8K sweep** (problems 247-499, running now)
2. **Synthetic casework generator**: SymPy-verified structured problems + a structure-encouraging teacher prompt; contrast batch so `split`/`abandon` appear in BC data
3. **Design-true regime**: API teacher + local small generator (Qwen 1.5B/3B); measure the small model's masked-action rate first
4. **Phase B**: per-node encoder + pointer head, BC warm-start on rung-1 trajectories, then Q-learning fine-tune against the verifier

One-sentence summary

Search and scale live in the teacher and the compute knob; what we *learn* is a tiny verifier-anchored controller over a frozen LLM.